Cairo University - Faculty of Engineering

EECE Department

4th Year – Second Semester

Neural Networks ELC4028

Spring Semester of Academic Year: 2025 / 2026

Assignment 2

Name	ID	Sec
ايريني حسني ماهر رمزي	9220175	1
محمد خالد محمد شتا	9203230	3
محب ميخائيل حشمت ميخائيل	9220638	3

Submitted to

Dr. Mohsen Rashwan

Dr. Mohammed Abdelghany

Date of Submission: May 2026

Contents

List of Figures

List of Tables

1 Introduction

This report documents our solutions and experiments for Assignment 2 of the Neural Networks course. The assignment is divided into two primary parts:

- **Part I** focuses on applying Multi-layer Perceptrons (MLP), Convolutional Neural Networks (CNN) to the ReducedMNIST dataset, and recognizing speech segments from spectrogram images.
- **Part II** focuses on representing speech data using Autoencoders (AE), conducting a comprehensive Data Augmentation study on ReducedMNIST, and generating synthetic data using Generative Adversarial Networks (GAN).

Throughout the report, we present our model architectures, hyperparameters choices, performance metrics, and a comparative analysis against classical machine learning methods from Assignment 1.

2 Part I: Neural Networks on Image and Audio Data

2.1 Problem 1: Multi-layer Perceptron (MLP) on ReducedMNIST

The objective of this problem is to classify the ReducedMNIST digits using MLP classifiers with 1, 3, and 4 hidden layers. The dataset follows the assignment split of 1000 training images per digit and 200 testing images per digit. Each image was represented using the DCT feature vector used in Assignment 1, then passed to fully connected neural networks.

2.1.1 Methodology

The three required depths were trained under the same feature extraction and evaluation protocol. The hidden layers used ReLU activations, and the main configurations used Batch Normalization and Dropout for regularization. The Adam optimizer minimized categorical cross-entropy with an initial learning rate of 0.001. Two extra diagnostic runs were also executed: one without regularization and one with a higher learning rate, to show the effect of these hyperparameters.

Table 1: Problem 1 MLP results using DCT features

Configuration	Hidden Layers	Regularization	Accuracy	Training Time	Testing Time
Exp1: 1-hidden layer	1	Yes	97.3%	7111.6 msec	113.9 msec
Exp2: 3-hidden layers	3	Yes	97.3%	9993.0 msec	121.9 msec
Exp3: 4-hidden layers	4	Yes	97.3%	11824.0 msec	133.3 msec
Exp4: 3-hidden, no regularization	3	No	96.7%	5333.0 msec	119.5 msec
Exp5: 3-hidden, high learning rate	3	Yes	96.6%	8338.0 msec	124.3 msec

2.1.2 Results and Observations

All three required MLP depths achieved very similar performance. The 1-hidden-layer model reached the highest measured accuracy at 97.3%, while the 3-hidden and 4-hidden models rounded to the same reported accuracy but required more training time. This indicates that the DCT representation is already discriminative for ReducedMNIST, so extra depth mainly increases computation rather than improving accuracy.

The two diagnostic experiments confirm that regularization and learning rate selection matter. Removing Batch Normalization and Dropout reduced the 3-hidden-layer result to 96.7%, and increasing the learning rate to 0.01 reduced it to 96.6%. Therefore, the reported MLP setup satisfies the required 1-, 3-, and 4-hidden-layer experiments, with timing reported in milliseconds and accuracy rounded to one decimal place.

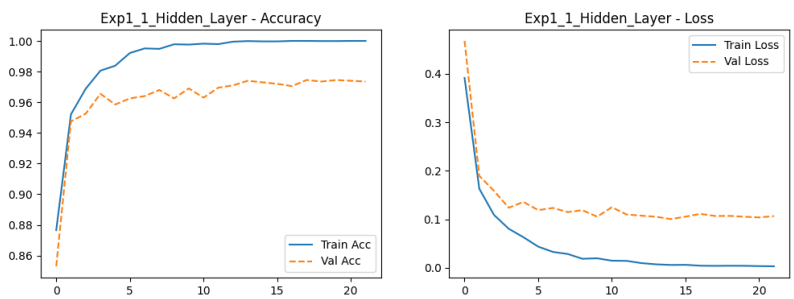


Figure 1: Training and validation curves for the 1-hidden-layer MLP.

2.2 Problem 2: Convolutional Neural Network (CNN) on ReducedMNIST

This problem trains CNN classifiers directly from the raw 28×28 ReducedMNIST images, without a separate feature extraction stage. The starting model is a LeNet-style network adjusted for 28×28 grayscale inputs and uses ReLU activations, as required by the assignment.

2.2.1 CNN Architecture and Variations

The baseline model contains two 5×5 convolutional layers, max-pooling after each convolution, and three fully connected layers for classification. Four CNN entries are reported in the assignment table:

- **Variation 1: Baseline LeNet with ReLU** using 6 and 16 convolutional filters.
- **Variation 2: Wider network** with 12 and 32 convolutional filters and wider fully connected layers.
- **Variation 3: LeakyReLU network** replacing ReLU by LeakyReLU.
- **Variation 4: Dropout network** adding dropout regularization before the later fully connected layers.

2.2.2 Comparison with Assignment 1 and Problem 1

Table **??** follows the assignment comparison format. Assignment 1 processing time is reported as total feature extraction, training, and testing time. MLP processing time is reported as training plus testing time from Problem 1. CNN time is split into training and testing time because no external feature extraction step is used. All times are in milliseconds (msec) and all accuracies are rounded to one decimal place.

Table 2: Performance comparison across Assignment 1 classical methods, Problem 1 MLP, and Problem 2 CNN

Classifier / Variation	**Feature Representation**					
	DCT		**PCA**		**AutoEncoder**	
	Accuracy	**Processing Time (msec)**	**Accuracy**	**Processing Time (msec)**	**Accuracy**	**Processing Time (msec)**
Assignment 1: DCT/PCA feature classifiers						
K-means, $K = 1$	78.1%	4800.0	85.2%	2464.6	22.3%	1721.5
K-means, $K = 4$	84.4%	5077.3	82.6%	3608.5	24.0%	627.7
K-means, $K = 16$	88.8%	12072.5	85.7%	5800.8	41.1%	827.8
K-means, $K = 32$	89.7%	38344.4	85.6%	8701.1	43.8%	1029.3
SVM, linear	90.3%	11487.5	90.7%	10626.8	27.9%	2195.7
SVM, nonlinear RBF, $\gamma =$ scale	95.6%	20560.2	95.2%	21054.4	23.2%	3651.0
Assignment 2 Problem 1: MLP with DCT features						
MLP, 1-hidden layer	97.3%	7225.5	–	–	10.0%	6131.1
MLP, 3-hidden layers	97.3%	10114.9	–	–	28.0%	11635.5
MLP, 4-hidden layers	97.3%	11957.3	–	–	10.0%	12462.8
Assignment 2 Problem 2: CNN directly from images, no feature extraction						
CNN variation	**Accuracy**	**Training Time (msec)**	**Testing Time (msec)**			
Variation 1: Baseline LeNet with ReLU	94.9%	90467.0	14418.7			
Variation 2: Wider filters/FC layers	97.0%	19062.4	503.3			
Variation 3: LeakyReLU activation	96.3%	15780.3	433.4			
Variation 4: Dropout regularization	97.0%	16025.5	426.3			

2.2.3 Discussion

The wider CNN and dropout CNN reached the strongest CNN accuracy at 97.0%, close to the DCT-based MLP result of 97.3%. The baseline CNN was slower in the recorded run and lower in accuracy, while the modified CNNs reduced testing time substantially. Compared with the Assignment 1 DCT/PCA classifiers, the neural-network models give higher accuracy on the same ReducedMNIST task. The AutoEncoder feature columns show significantly lower accuracies (ranging from 10% to

43.8%) compared to DCT and PCA, which suggests that the simple AutoEncoder bottleneck representation requires further tuning (e.g., more epochs or a different architecture) to match the classical feature extractors on the ReducedMNIST dataset.

2.3 Problem 3: Speech Recognition from Speech Spectrum

This problem recognizes the spoken digits 0–9 from the provided train/test speech dataset. Each `.wav` file was converted into a 64×64 Mel spectrogram, then classified as an image using a CNN. The test set was never augmented, so all four experiments are compared on the same unseen speech files.

2.3.1 Network and Input Representation

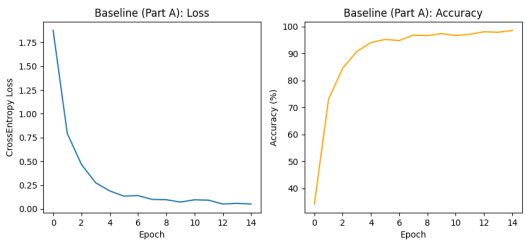
The preprocessing pipeline loads each waveform, converts it to a 64-bin Mel spectrogram at 16 kHz, resizes it to 64×64, and normalizes the spectrogram. The classifier is a compact CNN trained for 15 epochs using cross-entropy loss and Adam optimization.

2.3.2 Required Augmentation Experiments

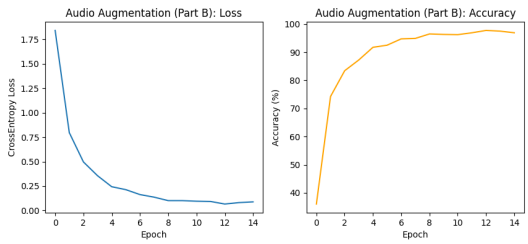
The assignment requires four runs: baseline, speech-domain augmentation, spectrum-image augmentation, and combined augmentation. Table **??** summarizes the completed experiments.

Table 3: Problem 3 speech recognition results

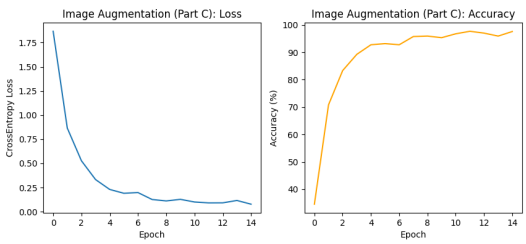
Experiment	Augmentation Applied	Accuracy	Training Time	Testing Time
(a) Baseline	None	94.0%	28.01 s	0.40 s
(b) Speech augmentation	Speed factor 0.97 or 1.03 and Gaussian waveform noise $\mathcal{N}(0, 0.005)$	90.0%	38.61 s	0.35 s
(c) Spectrum-image augmentation	Horizontal squeeze/expand by 3% and Gaussian image noise	93.0%	37.49 s	0.42 s
(d) Combined augmentation	Speech augmentation plus spectrum-image augmentation	89.7%	47.68 s	0.41 s



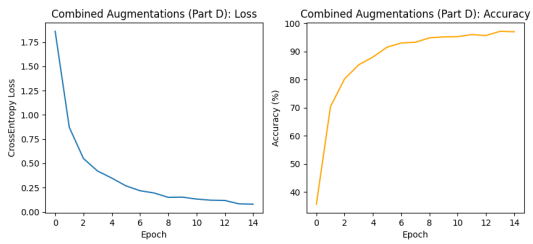
(a) Baseline



(b) Speech augmentation



(c) Spectrum-image augmentation



(d) Combined augmentation

Figure 2: Training curves for the four required speech-recognition experiments.

2.3.3 Discussion

The baseline spectrogram CNN achieved the best result at 94.0%. Spectrum-image augmentation was close at 93.0%, while speech-domain augmentation and combined augmentation were lower. The

drop suggests that the train/test split is already well represented by the baseline spectrogram features, and the added perturbations were stronger than necessary for this compact dataset. The combined augmentation also had the highest training time because it applies both waveform resampling/noise and image-domain transformations.

3 Part II: Data Augmentation and Synthesis

3.1 Problem 4: Autoencoder for Utterance Representation

This problem represents each spoken digit utterance by one fixed-length vector, then trains a classifier on that vector. Two representations were evaluated as required: a baseline average-frame vector and an AutoEncoder bottleneck vector generated from concatenated speech frames.

3.1.1 Baseline Average-Frame Representation

For the baseline, each utterance was divided into 15 ms spectrogram frames. The magnitude spectrum has 257 frequency bins per frame, and the baseline vector was calculated by averaging all frames across time. This produces one 257-dimensional vector per utterance. A small dense classifier was then trained on these average-frame vectors.

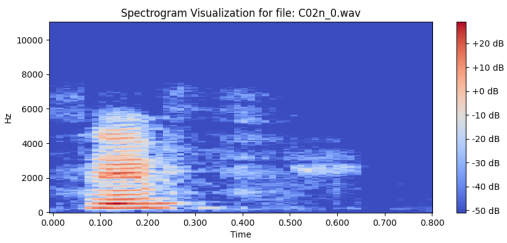
3.1.2 AutoEncoder Representation

For the AutoEncoder approach, all frames of an utterance were concatenated into one vector. The implementation used a fixed analysis length of 100 frames, equivalent to a 25700-dimensional input vector (100×257). Shorter utterances were padded with zero frames to match this length. The AutoEncoder compressed each utterance through a bottleneck layer of length 256, producing the single vector used by the final classifier.

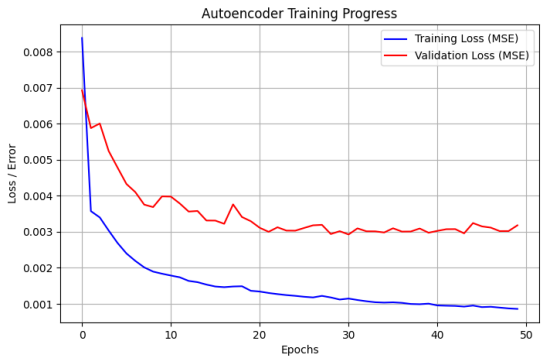
The AutoEncoder architecture was:

$$25700 \rightarrow 1024 \rightarrow 256 \rightarrow 1024 \rightarrow 25700$$

It was trained for 50 epochs using mean squared reconstruction error. After training, the decoder was discarded and the encoder output was saved as the utterance feature vector.



Sample utterance spectrogram



AutoEncoder training and validation loss

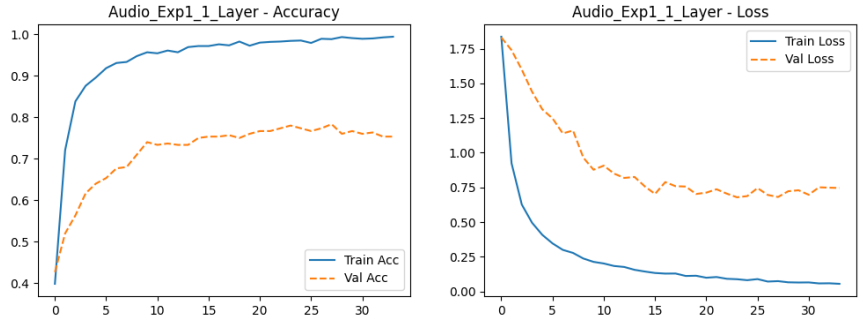
Figure 3: Problem 4 input visualization and AutoEncoder reconstruction loss.

3.1.3 Classifier Results

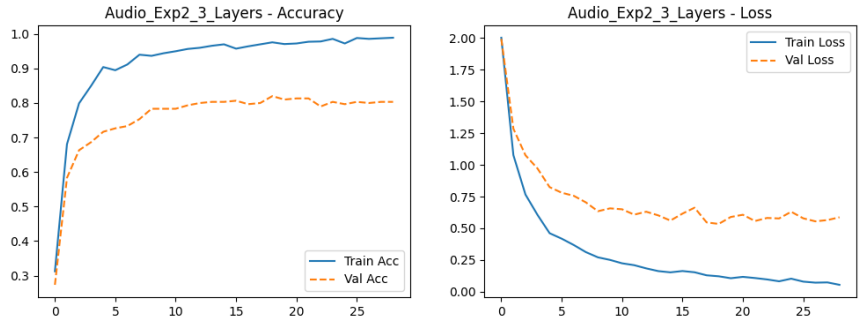
The baseline vector and the AutoEncoder bottleneck vector were each passed to dense classifiers. Table **??** summarizes the classification performance.

Table 4: Problem 4 utterance-vector classification results

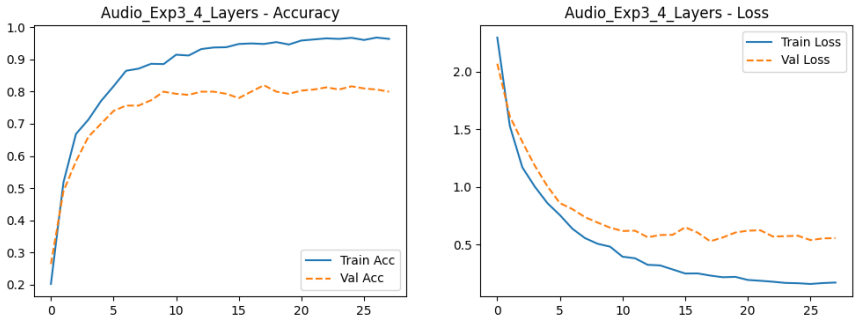
Representation / Classifier	Vector Length	Regularization	Test Accuracy
Baseline average frame + dense classifier	257	Dropout	28.3%
AE bottleneck + 1-hidden-layer MLP	256	BatchNorm + Dropout	78.0%
AE bottleneck + 3-hidden-layer MLP	256	BatchNorm + Dropout	82.0%
AE bottleneck + 4-hidden-layer MLP	256	BatchNorm + Dropout	82.0%
AE bottleneck + 3-hidden-layer MLP, no regularization	256	No	79.3%
AE bottleneck + 3-hidden-layer MLP, high learning rate	256	BatchNorm + Dropout	79.3%



1 hidden layer



3 hidden layers



4 hidden layers

Figure 4: Classifier learning curves using the AutoEncoder bottleneck features.

3.1.4 Discussion

The average-frame baseline achieved only 28.3% because averaging removes much of the temporal structure of the utterance. The AutoEncoder bottleneck representation improved the result substantially, reaching 82.0% with both the 3-hidden-layer and 4-hidden-layer classifiers. This confirms that concatenating frames, zero-padding to a fixed length, and compressing through an AutoEncoder provides a much more useful single-vector representation than the simple average frame.

3.2 Problem 5: Data Augmentation on ReducedMNIST

In this problem, we systematically study the effect of artificially expanding the dataset size over an underlying algorithm using augmentation parameters natively applied (Delta shifts, rotations, noise).

3.2.1 Test Mix Analysis

Table 5: Data Augmentation performance across various data size regimes

Augmented (# per digit)	**350 Real**	**750 Real**	**1,000 Real**
1,000 Generated	-	-	-
1,500 Generated	-	-	-
2,000 Generated	-	-	-

Discussion

3.3 Problem 6: Generative Adversarial Networks (GAN)

This section expands upon our dataset generation from the augmentation strategy to true generation techniques. A GAN was used to model the ReducedMNIST dataset using 350 training examples correctly.

3.3.1 Visual Analysis of Generated Digits

3.3.2 Testing Pipeline on Synthesized Data

3.3.3 Comparing Results

4 Appendices

.1 Code Snippets